

A Conceptual Metamodel to Bridging Requirement Patterns to Test Patterns

Taciana Novo Kudo*
Departamento de Computação
Universidade Federal de São Carlos
São Carlos, São Paulo, Brazil
taciana@dc.ufscar.br

Renato F. Bulcão-Neto
Instituto de Informática
Universidade Federal de Goiás
Goiânia, Goiás, Brazil
rbulcao@ufg.br

Auri M. R. Vincenzi
Departamento de Computação
Universidade Federal de São Carlos
São Carlos, São Paulo, Brazil
auri@dc.ufscar.br

ABSTRACT

Requirement patterns represent an abstraction of an application's behaviors and services that, in turn, may be replicated in similar applications. However, there has been a lack of efforts exploiting the benefits of requirement patterns in other phases of the software development life cycle, besides the requirements engineering itself. To address this gap, we propose the Software Pattern MetaModel (*SoPaMM*) that bridges requirement patterns to groups of scenarios with similar behaviors in the form of test patterns. *SoPaMM* allows the description of the behavior of a requirement pattern through a time executable and easy-to-use language aiming at the automatic generation of test patterns. Using *SoPaMM*, we model and implement a behavior-driven functional requirement pattern for a web-based user authentication application. Our preliminary results point out that a requirement pattern can be an executable specification capable of generating automated tests.

CCS CONCEPTS

• **Software and its engineering** → **Requirements analysis; Reusability; Acceptance testing**; • **Computing methodologies** → *Modeling methodologies*.

KEYWORDS

requirement pattern, test pattern, reuse, behavior, metamodeling

ACM Reference Format:

Taciana Novo Kudo, Renato F. Bulcão-Neto, and Auri M. R. Vincenzi. 2019. A Conceptual Metamodel to Bridging Requirement Patterns to Test Patterns. In *XXXIII Brazilian Symposium on Software Engineering (SBES 2019), September 23–27, 2019, Salvador, Brazil*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3350768.3351300>

1 INTRODUCTION

There is a consensus among researchers and software industry professionals about the value of requirements engineering and the

critical vulnerability of software projects whenever requirements-related activities are poorly executed. Most software projects extrapolate budget and delivery times due to incorrect, omitted, misinterpreted, or conflicting requirements [3, 21].

In the last decade, the practice of requirements reuse [2, 7] has been a feasible alternative to mitigate those issues, assisting requirements engineers to produce better quality specifications. A fairly discussed reuse approach is the requirement pattern (RP) concept, which is an abstraction that aggregates behaviors and services observed in multiple applications that can be reused in similar software applications [23]. Several studies demonstrate that RPs can promote the completeness of requirement specifications and the increase of the productivity of the development team, among other benefits [3, 4, 8, 14, 23].

However, there have been few research efforts on the applicability of RPs into other phases of the software development life cycle (SDLC), disregarding the overall impact of requirements engineering as reported in a previous work [9]. As an example of the influence of requirements activities in the SDLC, the popular V-model [18] associates a user acceptance testing phase for the requirement analysis phase to determine whether a software system satisfies the requirements specified.

As another type of software pattern, a test pattern (TP) describes generic solutions to test common recurrent behaviors [12]. TPs help a tester understand the context of a testing practice as well as decision making among alternative patterns [11]. Repetitive, alike, and high-value test practices are likely candidates to be documented as TPs.

Therefore, by considering both the intrinsic relation between requirements engineering and testing and the lack of investigations into the advantages of RPs at other phases of the SDLC [9], we suggest that an RP can be related to a TP as well as be tested more assertively if test cases are documented into that TP.

This paper presents a conceptual metamodel called *Software Pattern MetaModel (SoPaMM)*, which defines how requirement and test patterns can be written, organized, related, and classified. For this, *SoPaMM* reuses concepts and practices of the Behavior-Driven Development (BDD) agile methodology [1, 15] and the design pattern called Page Object Model (POM) [10, 20].

We show the applicability of *SoPaMM* through the modeling and implementation of a functional requirement pattern for user authentication into a web application integrated with automatically generated user acceptance test cases. Our preliminary results demonstrate the advantages of using *SoPaMM*: a comprehensive and structured way of combining, extending and reusing RPs and

*Currently, Mrs. Kudo is a Ph.D. candidate at the Universidade Federal de São Carlos and also works as an assistant professor at the Universidade Federal de Goiás.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBES 2019, September 23–27, 2019, Salvador, Brazil

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7651-8/19/09...\$15.00

<https://doi.org/10.1145/3350768.3351300>

TPs, the automated testing of RPs, and the decoupling of user interface elements from application test cases.

The structure of this paper is organized as follows: Section 2 discusses related work; Section 3 describes the conceptual metamodel proposed; Section 4 presents an instance model of *SoPaMM* implemented into a web application; and Section 5 brings our concluding remarks and future work.

2 RELATED WORK

Based on a recent previous work [9], there is little research on the integration of RPs with artifacts produced in other SDLC phases. For this reason, this section describes research endeavors that combine metamodeling and at least one of the types of software patterns covered in this paper.

Some studies propose modeling of RPs using metamodels, but with no integration with other SDLC artifacts [4, 24]. As one of the pioneering efforts on metamodeling of RPs, Franch et al. [4] define the structure of an RP, the possible relationships among RPs, and classification criteria for grouping them. The main idea behind the use of metamodeling is to provide more flexibility on how to model RPs by decoupling the types of RPs and allowing the types of relationships more configurable.

Another research on metamodeling and RPs, Badamasi et al. [24] present a metamodel that represents RPs with variability modeling and traceability of software artifacts. The main goal is to improve the systematic reuse of RPs by integrating concepts of software product line and model-driven engineering.

To the best of authors' knowledge, there is no work proposing the combined use of the metamodeling theory and TPs. The proposals of use of TPs range from unit testing, integration testing, and user interface testing [11, 12, 17]. Coelho et al. [17] define a TP for unit and integration testing of layered information systems. The main objective is that each test class focuses on testing the specific features implemented by each layer in an application. Meszaros [11], in turn, describes sixty-eight test patterns to facilitate how to write, understand, and maintain unit testing. The goal is to allow unit tests using TPs more robust and repeatable, and thus more cost-effective. Finally, Moreira and Paiva [12] develop six types of graphical user interface test pattern, aiming at elaborating generic test strategies for recurrent behaviors applicable over different applications. Those authors also develop a domain-specific language that supports the representation of such test patterns.

Despite not using metamodeling with patterns as other works do, we have found one study that integrates RPs with mutation tests [22]. Natural language-based requirement descriptions generate mutants, and every mutant is associated with a linear temporal logic formula that conveys its semantics. For each type of RP, the relevant potential faults are identified, and appropriate mutations are introduced. Formulas of mutants may have several purposes, such as test generation, and analysis of test case sets adequacy.

In comparison with those mentioned works above, the novelty here is the integration of RPs and TPs into a metamodel, where BDD concepts (feature, scenario, example) describe the behavior of an RP. This structured description of an RP can produce an executable and, at the same time, an easy-to-understand specification that enables the generation of TPs that can be automated.

3 THE SOPAMM CONCEPTUAL METAMODEL

The Software Pattern Metamodel (*SoPaMM*) describes how RPs and TPs are to be specified, related, stored, and classified. The underlying idea to *SoPaMM* is not only the reuse of an RP to be part of a software requirements specification, but also the further reuse of one or more TPs associated.

The *SoPaMM* metamodel borrows the MetaObject Facility's (MOF) metamodeling architecture [13] to make it easier the interoperability of the proposed model. The OMG's MOF defines a four-layered architecture that constitutes the foundation of the *SoPaMM* proposal, in which lower layer models are instances of immediately upper layer models, as illustrated in Figure 1.

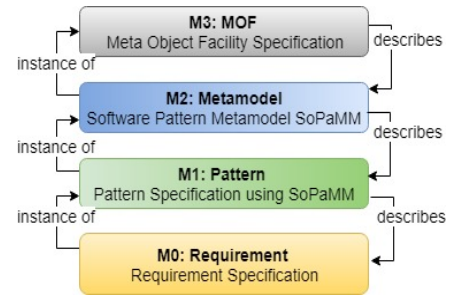


Figure 1: The integration between OMG's MOF and *SoPaMM*.

The MOF specification is in M3-layer and serves as a reference for the description of the *SoPaMM* metamodel located in M2-layer; hence, it is an instance of OMG's MOF. The M1-layer, in turn, describes requirement patterns and test patterns instanced from the *SoPaMM* metamodel. Finally, the M0-layer represents the description of a real-world requirement specification document, whose contents are instances of the M1-layer patterns.

Figure 2 depicts a UML class diagram of the *SoPaMM* metamodel, i.e., this corresponds to the M2-layer in Figure 1. Each *SoPaMM* concept — classes, attributes, and associations — is described next.

3.1 SoftwarePattern and SoftwarePatternBag

As its name suggests, the class *SoftwarePattern* (SP) represents the set of software patterns that can be supported by the *SoPaMM* metamodel. Software patterns research efforts and standardized specifications [4, 6, 8, 12, 23] have influenced the definition of the class SP. The definition of the class SP attributes is as follows:

- **Name:** the software pattern name (e.g., *LoginPasswordAuthenticationFRP*).
- **Author:** the software pattern author's name (e.g., John Doe).
- **Version:** the software pattern version date and number (e.g., version 1.0 of 2019/03/28).
- **Goal:** it describes what the software pattern is for (e.g., a functional requirement pattern for user authentication through login and password).
- **Source:** the source of information for the creation of the software pattern such as stakeholders' and similar software systems' names, and the identification of a physical or digital document, a standard specification, or law. For instance, consider the URL of the Brazilian Certification Manual for Electronic Health Record Systems (SBIS-CFM-2016-v4-2).

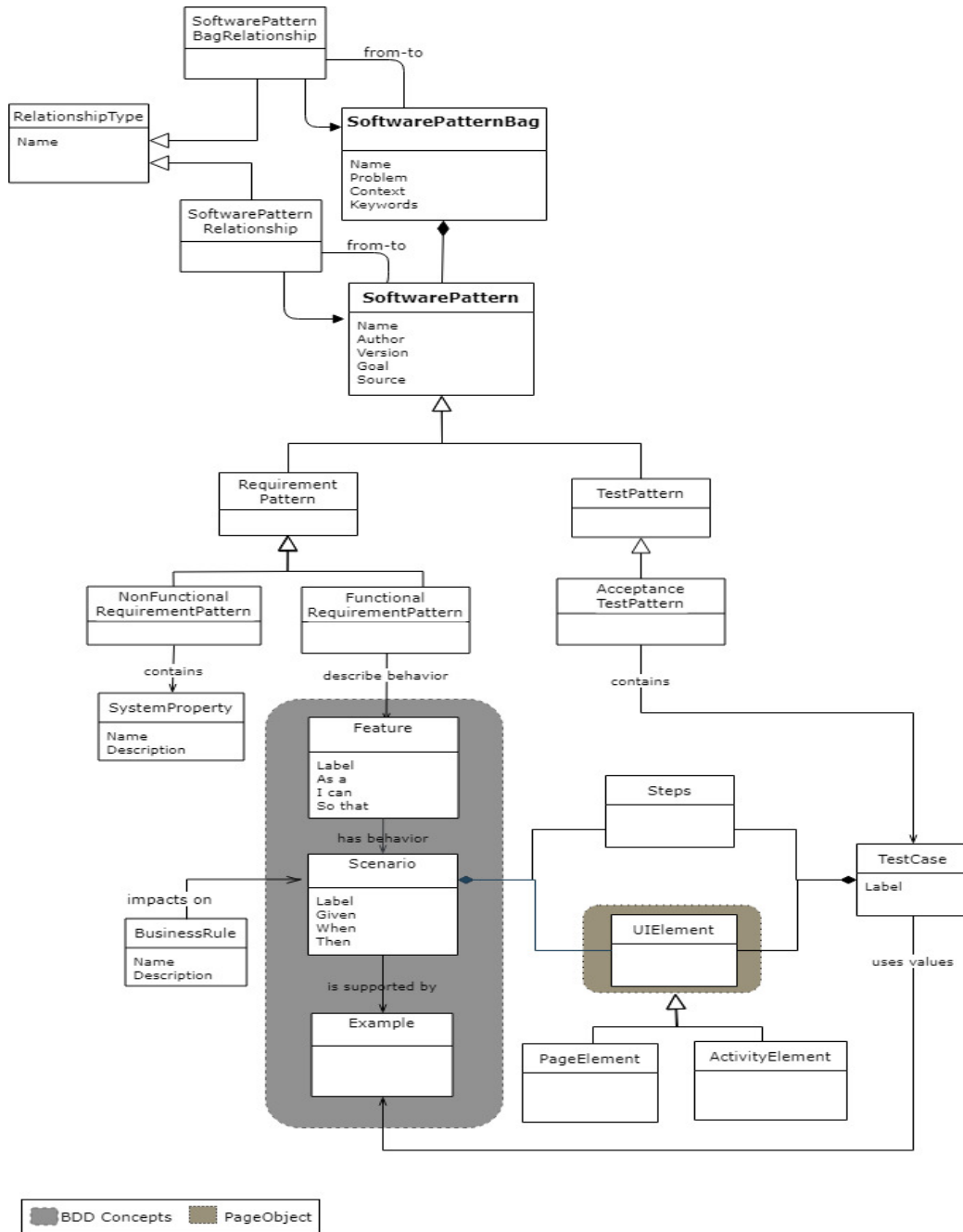


Figure 2: An overview of the SoPaMM metamodel.

In turn, the class *SoftwarePatternBag* (SPB) is a composition of one or more SPs. The goal is to allow not only the composition of multiple SPs but also of different SP types such as requirement patterns and test patterns. As currently modeled, the *SoPaMM* metamodel is extensible in terms of new types of SPs, e.g., by extending the superclass *SoftwarePattern* to include the class *Design Pattern*, with minimal impact on the structure defined. The definition of the class SPB attributes is as follows [5, 16]:

- *Name*: a sequence of characters that smoothly conveys the SPB contents to mitigate misinterpretation (e.g., *LoginPasswordAuthenticationSPB*).
- *Problem*: a statement of the problem the SPB addresses (e.g., only identified and authenticated users can get access to system data and functionalities).

- *Context*: the situation in which the problem occurs (e.g., a user must be able to access the system through login and password authentication).
- *Keywords*: terms for indexing and search purposes (e.g., authentication, login, and password).

3.2 Relationships of SP and SPB

An SPB may be related to other entities of the same type (e.g., an SPB is a *composition* of other SPBs, or *depends on* a particular SPB). Similarly, an SP may also be related to other SPs (e.g., an SP *depends on* another SP, or *uses* information from another SP).

Although the literature has defined relationships types among software requirement patterns such as *extends*, *has*, and *uses* [4, 23], a higher diversity of types may exist according to applications' context. For this reason, both relationships of SPBs and SPs are not pre-defined in *SoPaMM*, aiming thus metamodel flexibility. The classes *SoftwarePatternBagRelationship* and *SoftwarePatternRelationship* allow this flexibility through the superclass *RelationshipType*.

3.3 Requirement Pattern Definitions

Back to the definition of the class *SoftwarePattern* (SP), this specializes in two classes until this moment: the classes *RequirementPattern* (RP) and *TestPattern* (TP). The class RP, in turn, specializes in the classes *NonFunctionalRequirementPattern* (NFRP) and *FunctionalRequirementPattern* (FRP). As subclasses of SP, the classes RP, NFRP, FRP, and TP share the same SP attributes and differ each other regarding their relationships with the other *SoPaMM* classes.

Nonfunctional requirement patterns (NFRP) relate to software system properties (the class *SystemProperty*) described by textual attributes (i.e., name and description). For instance, user credentials must be ever validated by an authentication server, forbidding user authentication on the client-side.

Different from related works, the modeling of functional requirement patterns (FRP) in *SoPaMM* is influenced by concepts of the Behavior-Driven Development (BDD) methodology [1]. The class *Feature* describes FRPs' behaviors through a high-level and straightforward user story syntax (As <stakeholder>, I CAN <what?> SO THAT <why?>).

The class *Feature* also groups related *Scenarios* supported by one or several *Examples*. Scenarios make use of the Gherkin language syntax [19] (GIVEN <an initial context> WHEN <an event occurs> THEN <an expected outcome>), which allows a concise description of multiple examples with different values for a scenario. The concepts *Feature*, *Scenario*, and *Example* give not only structure and meaning to FRPs' behavior but also automate the respective test specifications. *SoPaMM* also allows the description of business rules that impact on an FRP's behavior.

3.4 TestPattern Definitions

Until this moment, the *TestPattern* (TP) superclass specializes in the class *AcceptanceTestPattern* (ATP), which contains one or more elements of *TestCase* which, in turn, use input data of the class *Example* of a scenario.

The way we integrate FRPs and ATPs is through the modeling of each FRP scenario as a composition of steps and *UIElement* object types. Steps are testable abstractions of the steps (GIVEN, WHEN, and

THEN), which are characteristic of BDD scenarios, while *UIElement* represents user interface elements. Besides, a *TestCase* related to an ATP is also described as a composition of steps and user interface elements, and makes use of data values of the class *Example* for running tests.

The class *UIElement* demonstrates the concept of the *Page Object* design pattern [10, 20], which reduces the coupling between user interface elements and test cases. This way, we promote reusability and also facilitate test cases maintenance. The *UIElement* can be of two types: *PageElement* for web applications, or *ActivityElement* for mobile applications.

4 PROOF OF CONCEPT

The first step of this proof of concept includes the development of a *SoPaMM*-based M1 model for user authentication through login and password, which is a recurrent functional requirement in most software systems. The structure of the *UserAuthLoginPassword:SPB* (*SoftwarePatternBag*) in Figure 3 is described next.

- a composition of one *FunctionalRequirementPattern* (FRP) and one *AcceptanceTestPattern* (ATP);
- the *UserAuthLoginPassword:FRP* includes one feature *LoginPassword:Feature* that, in turn, has two possible behaviors modeled as scenarios, i.e., *SuccessfulLogin:Scenario* and *UnsuccessfulLogin:Scenario*;
- the classes *SuccessfulLogin:Example* and *UnsuccessfulLogin:Example* encapsulate data fields and test data to support the scenarios above, respectively; and
- the *UserAuthLoginPassword:ATP* contains two *TestCases* (i.e., *SuccessfulLogin:TestCase* and *UnsuccessfulLogin:TestCase*) that, in turn, reuse the same classes *Example* previously defined for the FRP.

Notice that the integration between the FRP and the ATP occurs in the composition of steps and page elements, whose modeling decision is shared between scenarios and respective test cases. Besides, the reuse of the classes *Example* between scenarios and test cases co-related strengthens this FRP-ATP integration.

For instance, both the classes *UnsuccessfulLogin:Scenario* and *UnsuccessfulLogin:TestCase* reuse the same classes *UnsuccessfulLogin:Steps*, *UnsuccessfulLogin:PageElement*, and *UnsuccessfulLogin:Example*. The same occurs with the class *SuccessfulLogin:Scenario*. This way, test cases are pre-defined in a requirement pattern and are up for running when the requirement is implemented. Therefore, the more the examples support the requirement, the higher the number of acceptance test cases and, consequently, the wider the coverage test in the implementation.

The second step of this proof of concept is the implementation¹ of a web-based user authentication application supported by the M1 model in Figure 3. The main goal is to demonstrate the automation of acceptance tests associated with functional requirement patterns.

The starting point is the implementation of the *Login.feature* file: it contains a user story description of the feature of the *UserAuthLoginPassword:FRP*, a Gherkin-based definition of the particular scenarios for successful and unsuccessful login behaviors, and test input data (or examples) defined in the M1 model are used for each corresponding scenario.

¹Apache Maven, the JUnit framework, the Selenium WebDriver, among others.

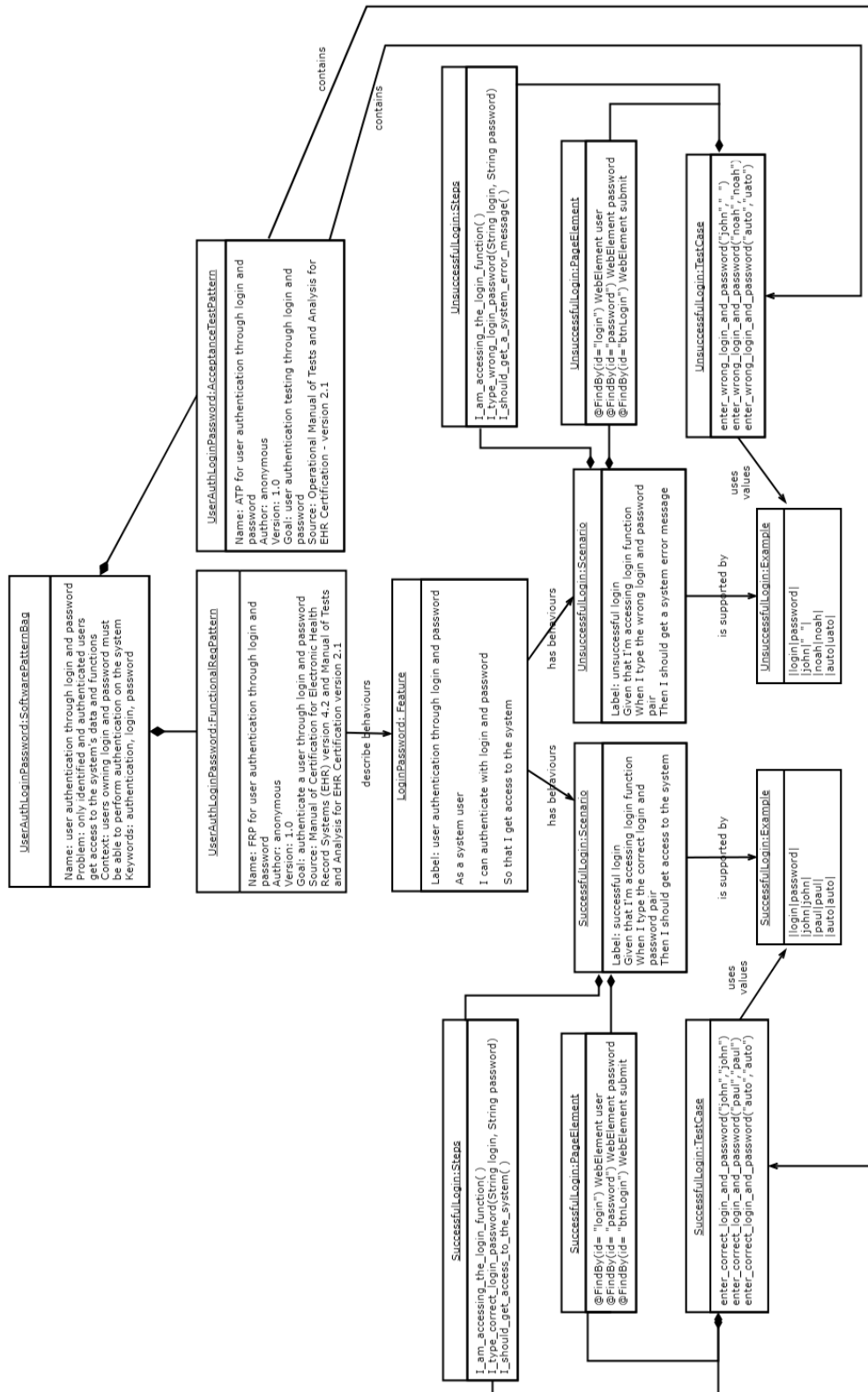


Figure 3: An example of an M1 model for the SoPaMM metamodel.

The class *LoginSteps* implements the steps (Given, When, Then) of each scenario (successful and unsuccessful login), whereas the class *LoginElementMap* represents the page elements for user authentication (i.e., text fields for login and password, and the submit button). The key point is the class *LoginPage* that extends *LoginElementMap* and associates the page elements with the steps of each scenario and the respective examples defined in *Login.feature*.

As a result, 18 steps were successfully tested for the user authentication FRP: 9 for each scenario (i.e., (un)successful login). We are aware that there is a tradeoff between test automation and the burden on the specification of RP with behavior (feature, scenario, and example). However, we advocate that test automation of RP with behavior may facilitate analysts', developers', and testers' tasks as well as the traceability between requirements and test cases.

5 FINAL REMARKS

Current trends in requirements engineering combine agility in the development process, the need for addressing emergent requirements, collaborative and test-oriented development, among others. To achieve these needs, requirements activities (e.g., acquisition and specification) have been changing in the past few years.

In that context, we propose the *SoPaMM* metamodel as a structured way of integrating requirement patterns and test patterns through concepts inherited from the behavior-driven development methodology. We describe a proof of concept in which we built a *SoPaMM*-based integrated model of RP and TP for user identification and authentication through login and password. In sequence, we implemented a simple web application with user access control to demonstrate the use in practice of that instance of *SoPaMM*. As a result, acceptance tests are automatically executed once the user identification and authentication is implemented.

The potential benefits of *SoPaMM* are as follows:

- verifiability – an RP is an executable specification capable of generating automated tests;
- ease of communication – the specification of an RP makes use of well-known and easy-to-use languages (user story and Gherkin syntax) for several stakeholders;
- extensibility – the metamodel has extensible points for the definition of new types of relationships between RPs and TPs; the same applies to the support for other types of software patterns (e.g., unit testing patterns);
- separation of concerns – test cases and UI elements are modeled apart to ease the reuse and maintenance of UI test cases.

Currently, we have been working on two lines of investigation: (i) the creation of a catalog of behavior-driven RPs from a set of requirements used in the certification process of electronic health record (EHR) systems in Brazil; and (ii) the development of a software tool to facilitate the use of *SoPaMM*-based patterns in real-world software projects.

ACKNOWLEDGMENTS

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001. Renato Bulcão-Neto is grateful for the scholarship granted by CAPES/FAPEG (88887.305511/2018-00), as postdoctoral intern at the Dept. of Computing and Mathematics, FFCLRP-USP.

REFERENCES

- [1] David Chelmsky, Dave Astels, Bryan Helmkamp, Dan North, Zach Dennis, and Aslak Hellesoy. 2010. *The RSpec Book: Behaviour Driven Development with Rspec, Cucumber, and Friends* (1st ed.). Pragmatic Bookshelf, Raleigh, NC.
- [2] Yuri Chernak. 2012. Requirements Reuse: The State of the Practice. In *2012 IEEE International Conference on Software Science, Technology and Engineering, SWSTE 2012, Herzlia, Israel, June 12-13, 2012*. IEEE Computer Society, Los Alamitos, CA, USA, 46–53.
- [3] Xavier Franch. 2015. Software Requirements Patterns: A State of the Art and the Practice. In *Proceedings of the 37th International Conference on Software Engineering - Volume 2 (ICSE '15)*. IEEE Press, Piscataway, NJ, USA, 943–944.
- [4] Xavier Franch, Cristina Palomares, Carme Quer, Samuel Renault, and Francois De Lazzer. 2010. A Metamodel for Software Requirement Patterns. In *Requirements Engineering: Foundation for Software Quality*, Roel Wieringa and Anne Persson (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 85–90.
- [5] Cecilia Haskins. 2003. Using Patterns to Share Best Results - A proposal to codify the SEBOK. *INCOSE International Symposium* 13, 1 (2003), 15–23.
- [6] IEEE. 2018. ISO/IEC/IEEE 29148:2018 International Standard - Systems and software engineering - Life cycle processes - Requirements engineering. *ISO/IEC/IEEE 29148:2018(E)* 2 (2018), 1–92.
- [7] Mohsin Irshad, Kai Petersen, and Simon Poulding. 2018. A Systematic Literature Review of Software Requirements Reuse Approaches. *Inf. Softw. Technol.* 93, C (Jan. 2018), 223–245.
- [8] Sascha Konrad and Betty H.C. Cheng. 2002. Requirements patterns for embedded systems. In *Proceedings IEEE Joint International Conference on Requirements Engineering*. IEEE, Essen, Germany, 127–136.
- [9] Taciana N. Kudo, Renato F. Bulcão-Neto, Alessandra A. Macedo, and Auri M.R. Vincenzi. 2019. Padrão de Requisitos no Ciclo de Vida de Software: Um Mapeamento Sistemático. In *XXII Ibero-American Conference on Software Engineering, CIBSE 2019*. Curran Associates, Inc., New York, USA, 420–433.
- [10] Maurizio Leotta, Diego Clerissi, Filippo Ricca, and Cristiano Spadaro. 2013. Improving Test Suites Maintainability with the Page Object Pattern: An Industrial Case Study. In *ICST Workshops*. IEEE Computer Society, Washington, DC, USA, 108–113.
- [11] Gerard Meszaros. 2006. *XUnit Test Patterns: Refactoring Test Code*. Prentice Hall PTR, Upper Saddle River, NJ, USA.
- [12] Rodrigo M. L. M. Moreira and Ana C. R. Paiva. 2014. A GUI Modeling DSL for Pattern-Based GUI Testing - PARADIGM. In *ENASE 2014 - Proceedings of the 9th International Conference on Evaluation of Novel Approaches to Software Engineering, Lisbon, Portugal, 28-30 April, 2014*. IEEE, Lisbon, Portugal, 126–135.
- [13] OMG. 2002. Meta Object Facility (MOF) Specification, Version 1.4. *Object Management Group, Inc.* (2002).
- [14] Cristina Palomares, Carme Quer, and Xavier Franch. 2017. Requirements reuse and requirement patterns: a state of the practice survey. *Empirical Software Engineering* 22, 6 (2017), 2719–2762.
- [15] Lauriane Pereira, Helen Sharp, Cleidson de Souza, Gabriel Oliveira, Sabrina Marczak, and Ricardo Bastos. 2018. Behavior-driven Development Benefits and Challenges: Reports from an Industrial Study. In *Proceedings of the 19th International Conference on Agile Software Development: Companion (XP '18)*. ACM, New York, NY, USA, Article 42, 4 pages.
- [16] Linda Rising. 1999. Patterns: A way to reuse expertise. *IEEE Communications Magazine* 37, 4 (April 1999), 34–36.
- [17] Arndt von Staa Roberta Coelho, Uirá Kulesza and Carlos Lucena. 2005. The Layered Information System Test Pattern. In *Fifth Latin American Conference on Patterns Languages of Programming (SugarLoafPlop05)*. Campos do Jordão - Brasil, 1–16.
- [18] Paul Rook. 1986. Controlling software projects. *Software Engineering Journal* 1 (02 1986), 7.
- [19] John Ferguson Smart. 2014. *BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle* (first ed.). Manning Publications.
- [20] Andrea Stocco, Maurizio Leotta, Filippo Ricca, and Paolo Tonella. 2015. Why Creating Web Page Objects Manually if It Can Be Done Automatically?. In *Proceedings of the 10th International Workshop on Automation of Software Test (AST '15)*. IEEE Press, Piscataway, NJ, USA, 70–74.
- [21] Steve Tockey. 2015. Insanity, Hiring, and the Software Industry. *Computer* 48 (11 2015), 96–101.
- [22] Mark Trakhtenbrot. 2017. Mutation Patterns for Temporal Requirements of Reactive Systems. In *Proceedings - 10th IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2017*. IEEE, 116–121.
- [23] Stephen Withall. 2007. *Software Requirement Patterns*. Microsoft Press, Redmond, Washington.
- [24] Badamasi Ya'u, Azlin Nordin, and Norsaremah Salleh. 2016. Software Requirements Patterns and Meta Model: A Strategy for Enhancing Requirements Reuse (RR). In *2016 6th International Conference on Information and Communication Technology for The Muslim World, ICT4M, Jakarta, Indonesia*, 188–193.